Excise: One-Shot Capability Extraction from Open Language Models

Arya Manjaramkar
arya@e3group.ai

Workshop preprint, June 2026 • *artifact:* `pip install excise`

Abstract

A deployed language model usually exercises one capability while paying for all of them. PRISM showed that a single capability can run on a small fraction of MLP channels, but located that fraction with a staged pipeline—adapt, attribute, select, re-train—mapping the budget frontier one costly point at a time. We collapse the pipeline into a single training run: learnable hard-concrete gates on every MLP channel train jointly with a low-rank adapter under a forward-KL leash to the frozen base model, while an adaptive controller lowers the sparsity target as fast as behavior permits and *generation probes*—not loss curves—decide when the floor is reached. On PRISM's arithmetic benchmark, one run reaches **100.9% recovery at 5% of MLP channels** and automatically finds a **2.9%-channel floor at 97% recovery**, surpassing the staged pipeline's hand-tuned best (91.3% at 5.05%) on both axes; the controller's automatic floors reach **1.2% of channels on every seed, and 0.7% given more steps** (91% verbatim reproduction) — compute-bounded rather than behavior-bounded. The method needs no labels—the target is the model's own outputs—generalizes to function calling on Qwen3-4B (76% verbatim reproduction at 40% of channels, against a 19% attribution baseline), and data is the dominant knob it exposes: regenerating a JSON-extraction substrate from 5× more-diverse prompts collapses its floor from 27.5% to **1.9% of channels at 97% fidelity**. It also exports a physically smaller model: deleting the masked channels and pruning the vocabulary to the capability's token support shrinks Qwen2.5-Math-1.5B from **1.54B to 0.17B parameters (9×)** with no loss relative to masking, verified per run. Three negative results shaped the design: distribution-level KL reads healthy while true recovery collapses, layer-granular gates let the adapter overfit silently, and a renormalized sparse KL—the obvious cache implementation—quietly trains the unmasked model away from the base it is supposed to anchor. Total compute for every result in this paper: under $10 on one consumer GPU.

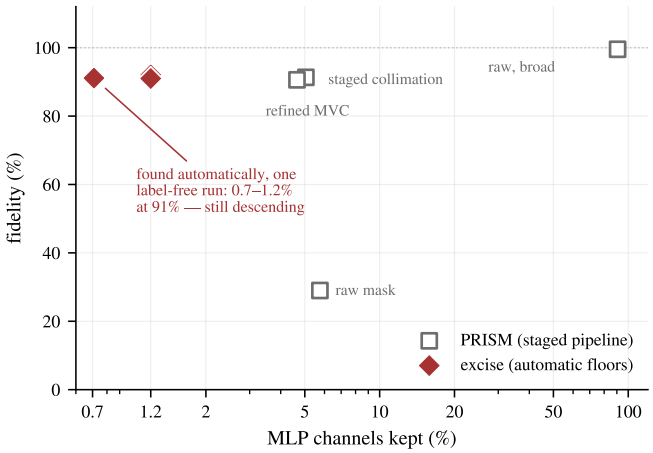

Figure 1: One run, no labels, a 6.5× smaller circuit than the hand-tuned pipeline. 2-digit addition on Qwen2.5-Math-1.5B. Gray squares: PRISM's staged pipeline as reported [13] (recovery, task accuracy). Red diamonds: `excise`'s automatically-found floors — three seeds at 1.2% of channels and one extended-budget run at 0.71% — plotted by held-out verbatim self-match, a stricter metric that lower-bounds recovery. Caveats for this comparison in §5.

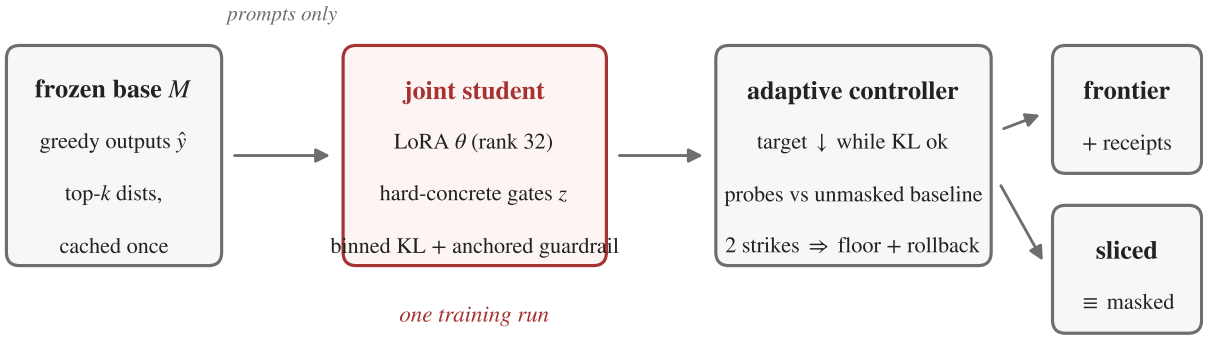

Figure 2: One-shot extraction. The frozen base model supplies label-free targets (greedy outputs and cached top-k distributions); gates and adapter train jointly under a KL leash; an adaptive controller paced by generation probes finds the floor; the run emits the sparsity–fidelity frontier, integrity receipts, and a physically sliced model.

1 Introduction

Model efficiency methods—pruning, distillation, quantization—ask how much of a network can be removed while *aggregate* performance survives. Mishra and Pagare [13] asked a sharper question: given one named behavior, how small a substrate of the network can carry it once everything else is switched off? Their answer, the PRISM framework, established what we adopt wholesale and credit as the foundation of this work: the *extraction contract* (a channel subset counts only if behavior survives with the complement deactivated), *recovery* as the metric, and the central finding that a behavior-preserving adaptation step ("collimation") can relocate a capability into far fewer channels than attribution alone suggests.

PRISM's pipeline, however, is staged: collimate, attribute with a relevance method, select a top-k mask, optionally re-collimate under the frozen mask. The stages create the central limitation their paper documents: an adapter trained *after* mask selection provably cannot shrink the mask, and one trained *before* can only move the frontier indirectly. The budget k is swept manually; each frontier point costs evaluations; the minimal circuit is found by hand-refinement. Their stated highest-value direction is to reshape the budget *during* training with an L_0 or gating objective.

We build exactly that, and package it as a tool. Figure 1 previews the headline comparison. Our contributions:

1. **One-shot joint extraction.** Hard-concrete gates [11] on all MLP intermediate channels train jointly with a LoRA adapter [10] under a mass-covering KL constraint to the frozen base. Selection and relocation negotiate continuously; the regime split between "collimate-then-attribute" and "attribute-then-collimate" dissolves (§2).
2. **Adaptive floor discovery with behavior probes.** A Lagrangian controller lowers the sparsity target whenever a KL budget is satisfied, and cheap greedy-generation probes veto the descent when the model's actual outputs degrade. The full sparsity–fidelity frontier and its floor come from one run (§2).
3. **Label-free operation.** The distillation target is the base model's own greedy output and full output distribution; no labeled data is required (§3).
4. **Physical export.** For gated MLPs, deleting masked channels is exactly equivalent to zero-masking them; pruning the vocabulary to the capability's token support compounds it. We verify both inside every run and ship them: 1.54B → 0.17B parameters (9×) at undamaged fidelity (§3.4).
5. **Calibrated negative results.** Distribution-level KL is an unreliable stopping signal at high sparsity; a renormalized sparse-KL cache silently trains the unmasked model away from the base it should anchor; and layer-granular gates permit silent train-distribution overfitting (§4). We argue these matter beyond our method: faithfulness metrics built on truncated logit divergence inherit the same blind spots.

2 Method

Setup. Let M be a frozen decoder-only transformer and C its MLP intermediate channels ($L \times d_{\text{ff}}$; the inputs to each block's down-projection), following PRISM's unit of extraction. Each channel c gets a hard-concrete gate $z_c \in [0, 1]$ parameterized by $\log \alpha_c$ [11, 12]; a LoRA adapter θ (rank 32, all linear modules) provides the relocation capacity. Gates initialize from a $|\text{grad} \times \text{act}|$ attribution percentile rather than a constant: attribution alone selects poor masks (Table 1), but it is a useful prior over the initial channel ordering. Given prompts x exercising the target capability, the teacher is the base model itself: greedy continuations $\hat{y} = M(x)$ and the full distribution $p_M(\cdot \mid x, \hat{y}_{<t})$ at each output position, cached once as top-128 sparse vectors *plus the residual off-support mass*. Keeping the residual is not bookkeeping: renormalizing the support—the obvious implementation—makes the KL read $-\log(\text{coverage})$ even when student and teacher agree exactly, and places positive gradient on every off-support token (§4).

Objective. Each step samples gates z (outside any gradient-checkpointed region; see §4) and minimizes

$$\mathcal{L} = \underbrace{\text{KL}(p_M \,\|\, p_{\theta,z})}_{\text{masked student tracks base}} + \beta_{\text{ce}}\,\text{CE}(\hat{y}) + \lambda\,\mathbb{E}[\|z\|_0]\,/|C| + \beta_g \underbrace{\text{KL}(p_M \,\|\, p_\theta)}_{\text{guardrail (every 4th step)}}, \tag{1}$$

with β_{ce}=0.05, β_g=1. Both KL terms are binned: the cached top-k support plus one residual bucket, which is zero exactly when the student matches the teacher. The forward (mass-covering) KL direction follows PRISM's collimation analysis: the student may not drop probability anywhere the base places it. The guardrail evaluates the *unmasked* adapted model and anchors it to the base, preventing the adapter from "solving the task masked" by becoming a different model; without it, unmasked accuracy silently drifted 3.7 points in our early runs. Because the train batch can only anchor points the adapter is already fitting, the guardrail alternates onto a rotating batch of off-task *anchor text* when provided, self-tunes its cadence from the measured anchor KL, and stays active through the polish phase — each of these closes a drift route we observed in practice (§3, §4).

Adaptive controller. A target open fraction τ starts at 1 and decays multiplicatively (×0.993 per step) whenever the EMA of the distillation KL is under a budget (0.02–0.025 in all experiments); the Lagrange multiplier λ follows $\lambda \leftarrow \max(0, \lambda + \eta\,(\bar{z} - \tau))$. Once the open fraction falls below a threshold, every 150 steps a *probe* hardens the current top-k mask and checks whether a greedy rollout would still reproduce the teacher outputs — computed as teacher-forced argmax agreement (exactly equivalent, by induction on the shared prefix, and ∼100× cheaper than generating), on a dev split excluded from the gradient batches so the floor is decided on non-memorized prompts. The masked reading is compared against the unmasked model measured *at the same step*, never against an assumed 100%: batched bf16 decoding alone costs several points of verbatim self-match, and adapter drift must not be misattributed to masking. If the gap exceeds tolerance twice consecutively, the floor is declared and the gates roll back to the last passing snapshot. After the descent stops, the mask is frozen at the floor budget and the adapter alone is polished briefly under the exact binary mask (guardrail still active), removing the stochastic-gate train/eval mismatch. Final evaluation hardens top-k masks at standard budgets; because the floor polish specializes the adapter to the floor mask (we measured off-floor budgets reading *below* the floor without correction), each off-floor budget is briefly re-polished from the floor-polished snapshot before its evaluation. The frontier this yields from a single run is a menu of deployable artifacts, not one point with decoration.

Receipts. Every run reports a random-mask control matched to the floor mask's per-layer channel counts (should be ≈0), unmasked drift, the base model's own decode-noise self-match (the honest ceiling for every other number), the probe and guardrail traces, and binomial confidence intervals on held-out fidelity. These guard the two failure modes we observed and one we inherited from PRISM's analysis: trivial tasks, cheating adapters, and masks that only correlate with behavior.

3 Results

All experiments ran on one RTX 4090 ($0.40/hr); the complete set below consumed under $10, through the released library's public interface — this is the entire workflow:

```
from excise import extract, load_sliced

result = extract("Qwen/Qwen2.5-Math-1.5B", prompts=prompts)  # no labels
print(result.report())            # frontier, floor, receipts, CIs
small = result.export_sliced(prune_vocabulary=True)  # 1.54B -> 0.17B
result.save("substrate/")         # receipts + the sliced artifact
model = load_sliced("substrate/sliced")              # round-trips
```

Code, configs, and per-run receipts: `github.com/Aryagm/excise`.

3.1 Arithmetic: surpassing the staged pipeline on its benchmark

We use PRISM's testbed: 2-digit addition on Qwen2.5-Math-1.5B [16] (canonical prompt `"a + b ="`, exhaustive sums, 10% held out; 250,880 MLP channels), zero-isolation deactivation, and recovery = masked accuracy / base accuracy (base: 97.4%). Table 1 gives the key points; Figure 1 situates the library's automatic floors against the staged pipeline.

Table 1: Arithmetic extraction. Every seed beats the staged pipeline at 5%, and every automatically-found floor is at or below the hand-refined minimal circuit with higher recovery. The unmasked guardrail held base accuracy exactly (97.4%) in all runs.

	channels kept	recovery
PRISM raw attribution mask	5.75%	29.0%
PRISM staged collimation	5.05%	91.3%
PRISM refined minimal circuit (manual)	4.65%	90.6%
random mask control (ours)	5%	0.0%
grad×act attribution mask (ours)	5%	0.3%
excise, one run (seed 42 / 43 / 44)	5%	**100.9 / 100.4 / 99.5%**
excise, automatic floor	**2.9 / 3.8 / 4.1%**	97.2 / 95.4 / 97.9%

Ablations (Figure 3): restricting the adapter to MLP projections (the strictest frozen-scaffold contract) costs ∼3 points at 5% and runs 25% faster; removing labels entirely (pure self-distillation, β_{ce}=0) still reaches 96.7% recovery but floors shallower (5.3%)—labels are not needed for validity, only for the last ∼2% of budget.

The floor is compute-bounded, not behavior-bounded. All three seeds descend to the controller's configured lower bound—**1.2% of channels**—with the masked–unmasked probe gap inside tolerance throughout, at 91.0–92.0% held-out verbatim self-match. Lowering the bound and giving one seed 9,000 steps reaches **0.71% of channels** (∼1,800 **of 250,880) at 91.1%**, exiting on the step cap while still descending at ∼0.1 percentage points per thousand steps (Figure 1). On saturated-coverage tasks the reported floor is set by adapter-compression compute, not behavioral collapse: the minimal circuit for 2-digit addition is somewhere below 0.7%, 6.5× under the hand-refined published one.

3.2 Function calling: a real-world capability, label-free

We extract single-turn function calling from Qwen3-4B [17] using BFCL prompts [14] (400 tasks; 391 yield clean tool calls from the base model; 320 train / 80 held out; 350,208 channels). No gold calls are used: the teacher is the model's own greedy tool call, and fidelity is *verbatim* self-match—stricter than BFCL's semantic scoring, since one diverging token fails the example. Held-out fidelity reaches 77.5% at 50% of channels and 76.3% at 40% (Figure 4). Two observations matter.

First, the *unmasked* adapted model self-matches only 86% on held-out prompts—masking is not the dominant loss; adapter drift under a 320-prompt training set is. Relative to that ceiling, the masked model retains ∼90% at 40–50% budgets.

Second, the gap between probe fidelity on training prompts (92–98% down to 12% open) and held-out fidelity quantifies a generalization phenomenon that exhaustively-covered tasks like arithmetic cannot exhibit; prompt diversity, not extraction machinery, is the binding constraint. A second experiment scales the prompts 6× and hardens the task (all six single-turn BFCL splits, plus 1–2 distractor tools per prompt so the substrate must carry tool *selection*): drift improves to 88.4% despite the harder task, and the binding constraint moves again—dev-split probes stop the descent

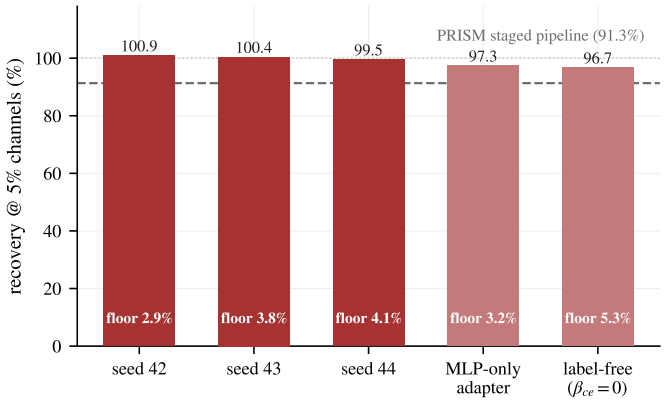

Figure 3: The result is seed-stable and survives ablation. Recovery at 5% of channels across seeds and ablations; channel floors found by the controller annotated within bars.

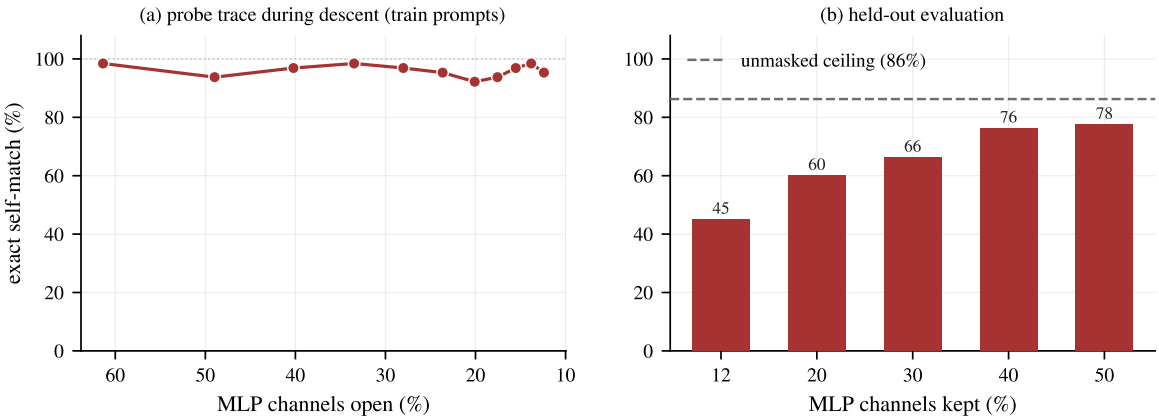

Figure 4: Function calling extracts label-free; data, not machinery, binds generalization. Qwen3-4B. Left: probe trace during descent — exact self-match holds at 92–98% from 61% open down to 12% open on training prompts. Right: held-out self-match by budget; the dashed line is the unmasked adapted model's own self-match (86%), which upper-bounds every masked number.

at 60.7% of channels (75.3% held) after only five epochs. With adequate data, function calling is *step*-bound, not memorization-bound; the two settings solve different tasks (one given tool vs. selection among distractors), so we do not compare them directly. For calibration: PRISM's raw mask recovered 19.1% at 36.2% of channels on the 8B sibling model, and their best staged collimator (with a curated, schema-validated gold corpus and a multi-run objective search) reached 84.6%; metric and model-size differences make this directional context, not a head-to-head (§5).

3.3 Breadth: other architectures and capabilities

To test that the method is a tool rather than a Qwen-arithmetic demo, we ran the *released library* (not the research scripts) on two further settings.

SmolLM2-1.7B (Llama architecture) [2]. Few-shot 2-digit addition extracts to a 7.4% floor at 97.2% held-out verbatim self-match; sliced 1.75B → 0.59B (3.0×); all receipts clean.

JSON extraction (Qwen2.5-1.5B-Instruct, chat-formatted prompts). From 3,000 label-free prompts spanning eight sentence forms and four instruction phrasings, structured JSON extraction floors at **1.9% of channels at 97.2%** held-out self-match—still step-capped—with 1.7 points of unmasked drift, a flat ~98% frontier at every budget, and a

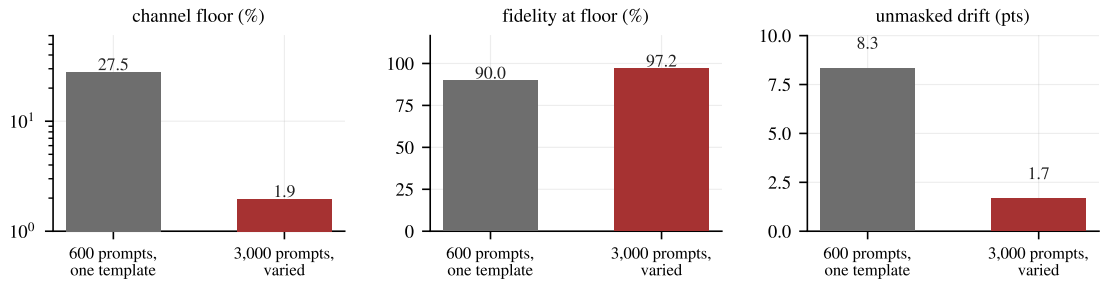

Figure 5: Same capability, same model, same config — only the prompts change. JSON extraction from 600 single-template prompts vs. 3,000 varied ones (eight sentence forms, four instruction phrasings): the floor collapses 14×, fidelity at the floor rises seven points, and unmasked drift nearly disappears. Both runs are label-free, so the diverse variant costs nothing but generation.

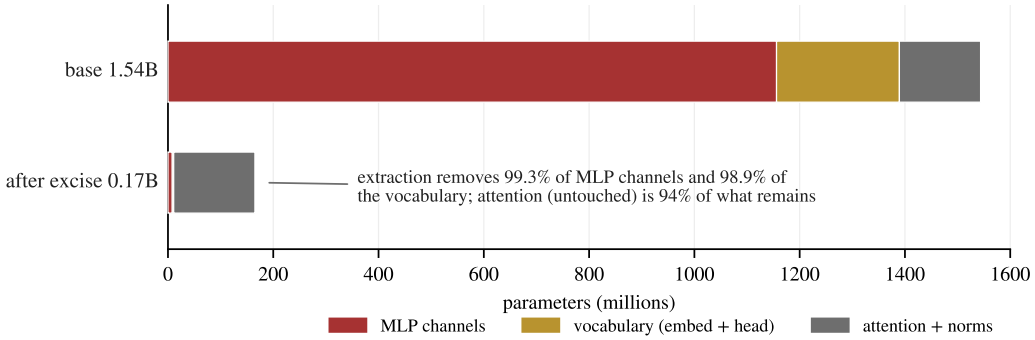

Figure 6: Where the parameters go. Qwen2.5-Math-1.5B before and after extraction at the 0.71% floor: 99.3% of MLP channels and 98.9% of the vocabulary are deleted outright; attention and norms, which the method does not touch, are 94% of what remains — the obvious next structured dimension.

verified 1.58B → 0.21B export (the capability touches 9,448 token ids; broader capabilities are broader in vocabulary too).

Data coverage is the dominant knob. Restricting the same capability to 600 prompts from a *single* sentence template inflates the floor 14× (to 27.5%) and drops fidelity to 90% (Figure 5)—and since held-out prompts share the template, even that 90% partly measures template fit. We initially read our settings as "capability breadth tracks substrate size"; this ablation corrects that. Substrate size measures the capability *as exercised by the prompts*: floors are upper bounds jointly set by data coverage and compression compute, not intrinsic capability sizes—and data is the cheapest lever, because label-free prompts cost nothing to diversify.

3.4 Physical export: slicing is masking, and the vocabulary is next

For gated MLPs, a zero-masked channel contributes nothing to the down-projection; a deleted one contributes nothing by absence. The two are mathematically identical, so the masked evaluations above transfer to a physically smaller model. We verify: slicing Qwen2.5-Math-1.5B at the 2.9% floor yields 94.8% accuracy vs. 94.7% masked (one example in 810), and **1.54B → 0.42B parameters (3.66×)**.

After MLP slicing, the dominant remaining cost is the vocabulary: 2-digit addition touches only 1,746 of 151,936 token ids, so slicing the embedding rows to that support (the exported model speaks a remapped id space, mapping stored in its config) takes the same model to **0.17B parameters (9.0×)** at the 0.71% floor (Figure 6). Both equivalences are re-verified inside every run and recorded in the receipts—masked vs. sliced vs. vocabulary-pruned self-match agreed to within one evaluation example here—and the library round-trips the heterogeneous-width artifact (`excise.load_sliced`).

Generation speedup in our original benchmark was only 1.11×, and an honest decode-throughput benchmark (batch

1–64, 64-token greedy decode, eager) shows *no* speedup at this scale: a 0.17B and a 1.54B model both decode at ~40 tokens/s/sequence on an RTX 4090, because single-stream decoding there is launch-overhead-bound, not compute-bound. Memory shrinks unconditionally; latency gains require larger models, batched serving, or a compiled runtime—we report this plainly because slicing claims are routinely overstated.

4 What fails, and what it implies

The method of §2 is an end state: most of its load-bearing choices exist because an earlier, more obvious implementation failed in a way worth documenting. This section is that record. Each failure was caught by the receipts the tool itself emits, and each implies something beyond this method.

Distribution-level KL is miscalibrated at high sparsity. Our first controller used the KL budget alone to pace the descent. It descended to 2.2% open with EMA KL comfortably under budget—while true recovery collapsed to 62–66% (Figure 8). Teacher-forced divergence on answer tokens simply does not capture compounding autoregressive degradation. The fix—periodically *generating* and exact-matching—is cheap (seconds per probe) and is the load-bearing stability feature of the method. We believe this observation extends beyond extraction: any faithfulness or circuit-quality metric built on logit divergence under teacher forcing inherits the blind spot.

Renormalized sparse KL quietly sharpens the model it should anchor. Caching the teacher's top-k probabilities *renormalized* — the obvious implementation, and our first — biases every loss built on it: with the student exactly equal to the teacher, the "KL" reads $-\log(\text{coverage})$ instead of zero, and its gradient $s_j - \hat{p}_j$ is positive on all off-support mass, actively pushing the student to concentrate onto each batch's cached support. On peaked tasks (arithmetic: coverage $\approx$1) the bias vanishes, which is how it survived three validated batteries. On diffuse tasks it surfaces in the worst place: the guardrail, whose entire job is to keep the unmasked model *at* the base, instead trains it to sharpen. The receipts caught the incident: a JSON run whose unmasked self-match read 55.8% (compounded by a then-guardrail-free polish phase). With the residual bucket restored, anchor batches, and the guardrail active through polish, the same task reads 88.3% against a 96.7% decode-noise ceiling — and its floor *improves*. Figure 7a shows why train-batch monitoring missed it: at the same step, the guardrail KL reads 3×10^{-4} on train batches (apparently perfect) and 5×10^{-3}–0.15 on off-task anchors. Drift lives precisely where a train-only guardrail cannot see it; an anchor that only watches the training distribution anchors nothing. The general lesson mirrors the probe lesson above: a truncated divergence is not a divergence, and the place it fails is precisely where its value is supposed to be zero.

Probes calibrated on memorized data flatter the floor. Our first release probed on prompts that were also in the gradient batches, against an assumed-perfect baseline—but even the unmodified base model reproduces its own batched-bf16 greedy targets only ~92% verbatim, so part of every probe reading was decode noise, not masking damage (Figure 7b shows the corrected decision rule). Re-running the identical extraction with dev-split probes scored against the unmasked model at the same step, rollback to the last passing sparsity, and the binned KL moves every number the tool reports (Table 2)—most of this paper's headline results were unreachable under the original calibration. One quantitative lesson generalizes: with n=64 verbatim probes, binomial noise alone is ±4 points, and a 3-point tolerance sat inside that band—the first two probes of a run tripped it, freezing the floor at 7.9%, where a noise-aware tolerance descends to 1.2% with probes passing throughout. Tolerances on generation metrics must be set against the metric's measured noise floor, not chosen for strictness.

Table 2: The same extraction under miscalibrated vs. calibrated receipts (arithmetic dogfood run; identical task, seed, hardware, and metric; base decode-noise ceiling 99.9%). Calibration is not cosmetic: it halves measured drift and reaches a 6× lower floor.

	original	calibrated (3 seeds)
floor found (probe-governed)	7.6%	**1.2 / 1.2 / 1.2%**
held-out self-match at floor	89.0%	**91.4 / 92.0 / 91.0%**
unmasked drift from base	10.7 pts	**6.2 / 5.8 / 7.0 pts**
exported parameters (from 1.54B)	0.48B	**0.17B**

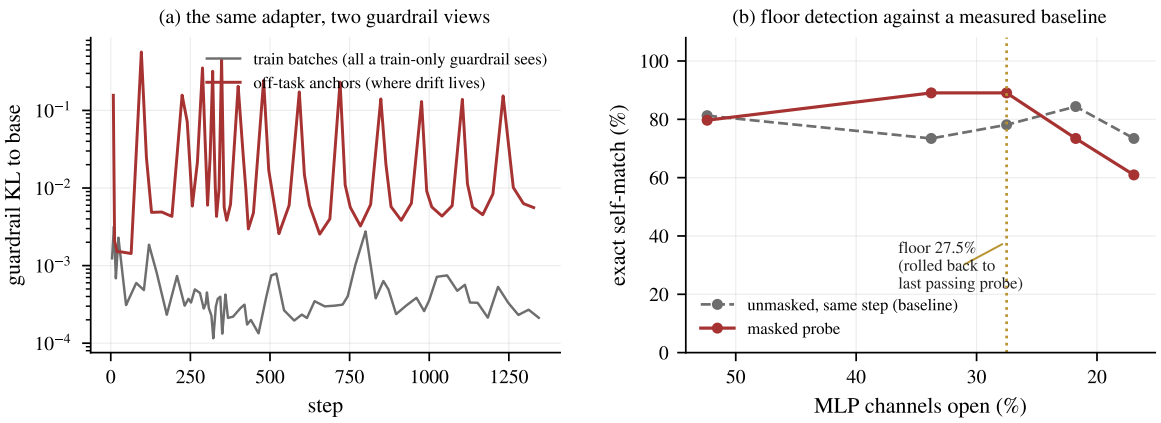

Figure 7: Two failure modes, instrumented (JSON extraction run). (a) The guardrail KL measured on train batches (gray) vs. rotating off-task anchor batches (red), same adapter, same steps: the train-batch view reads 10–100× lower than the anchor view. Drift lives exactly where a train-only guardrail cannot see it. (b) Floor detection: the masked probe (red) is compared against the unmasked model measured at the same step (gray, itself depressed by decode noise and residual drift), never against an assumed 100%; when the masked probe falls below tolerance twice, the gates roll back to the last passing sparsity.

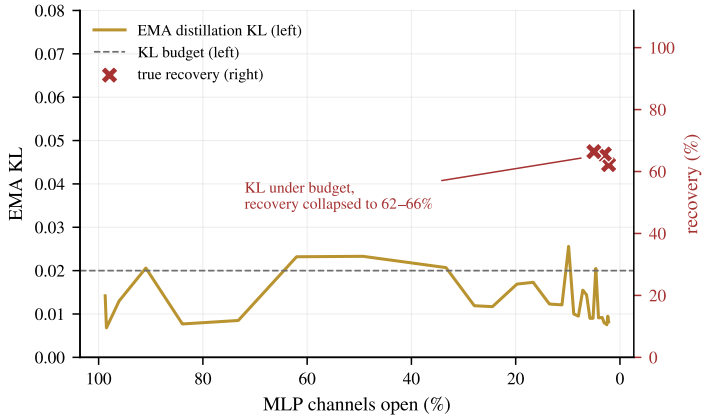

Figure 8: Losses lie; generations don't. An early KL-only controller: EMA distillation KL stays under budget throughout the descent (left axis) while true held-out recovery collapses (crosses, right axis). Generation probes detect this; losses do not.

Layer-granular gates permit silent overfitting. Adding per-layer gates (hierarchical L_0, intended to enable wall-clock-friendly whole-layer drops) closed 16 of 28 layers; training KL stayed low because the adapter rerouted around missing layers *on the training distribution*, and held-out recovery collapsed to 66%. Channel-granular gates with all layers present do not exhibit this. Structured sparsity for speed should be grafted on after extraction, not learned jointly.

Engineering notes. Stochastic gate sampling must occur outside gradient-checkpointed regions: in-region RNG produces a different recomputation graph and breaks backward. Pre-sampling per step costs nothing and composes with checkpointing, which 4B+ models require on consumer GPUs. Separately, grad×act attribution must accumulate under `no_grad`: an in-place `+=` of a grad-requiring product silently turns the accumulator into an autograd node retaining every iteration's activation graph—megabytes and invisible on 10-token arithmetic (it survived every validated battery), gigabytes per iteration and an instant OOM on 100-token JSON. Bugs that scale with sequence length hide in short benchmarks.

5 Honest comparison notes

- **Baselines.** Our arithmetic baseline reproduces PRISM's qualitative finding (attribution masks fail small: 0.3% recovery at 5%) but uses grad×act attribution and zero-isolation rather than their ReLP and counterfactual patching; PRISM's reported numbers come from their paper, not our re-runs. Zero-isolation is the harsher operator, so our baseline understates theirs.
- **Function calling.** Comparisons differ in model size (4B vs. 8B) and metric (verbatim self-match vs. BFCL exact match); we present them as context, not a head-to-head.
- **Seeds.** Arithmetic results are 3 seeds; function calling is a single seed, and its floor was step-cap-limited, not probe-detected.
- **Scope.** PRISM's 10^3 H100-hour budget bought breadth (translation, an 8B model, extensive controls) that our $10 did not; the claim here is that the *per-result* process cost collapses, not that our evidence is broader.

6 Related work

PRISM [13] defines the problem, contract, and metric we use; our method is their stated future direction, built. Learnable L_0 masks during adaptation descend from Louizos et al. [11] and movement pruning [15]; differentiable circuit discovery applies the same machinery to interpretability [3, 4]. Our distinction from both is the objective: a behavior-preservation contract with an unmasked-validity guardrail, rather than aggregate-loss compression or circuit faithfulness. Self-distillation into a substructure relates to on-policy and generalized knowledge distillation [1, 9], here with teacher and student sharing weights and the student differing only by a mask. Superposition [6] explains why attribution-only masks fail and adaptation is necessary; sparse autoencoders [5] and weight-sparse training [8] change the basis instead. The lottery-ticket line [7] seeks subnetworks that *retrain* to full performance; extraction requires the behavior to survive *without* retraining from initialization.

7 Limitations

Capabilities are bounded by prompt coverage: the tool can flag a generalization gap (via held-out receipts) but not close it without more data. Self-match undercounts semantic fidelity. Vocabulary-pruned artifacts speak a remapped id space, and the function-calling vocabulary support is 101k of 152k ids, so that lever pays little there. Attention is untouched and dominates the exported model. We have not yet run the 8B headline, multi-seed function calling, or non-Qwen replications beyond SmolLM2 and CPU-level architecture tests; these gate any stronger claims. Easier extraction also eases misuse (extracting narrow capabilities from safety-tuned models); receipts and intended-use documentation ship with the tool, mirroring PRISM's stance.

8 Conclusion

PRISM posed capability extraction as a contract—behavior must survive with the complement switched off—and named training-time budget reshaping as its highest-value open direction. We built that: one run, on one consumer GPU, that selects and relocates simultaneously, finds its own floor by generating rather than by watching losses, and leaves behind a frontier, integrity receipts, and a model 9× smaller. The headline numbers surpass the staged pipeline on its own benchmark, but the durable lesson may be the negative one: logit-level divergence is not behavior, and any metric built on it will miss collapses that a cheap generation probe catches. Extraction is now a `pip install`, not a pipeline.

Reproducibility

All numbers in this paper were produced by the released library or the research scripts in the same repository, on one rented RTX 4090. The paper's figures regenerate from committed result JSONs via `paper/make_figures.py`.

References

[1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

[2] Loubna Ben Allal et al. Smollm2: When smol goes big – data-centric training of a small language model. *arXiv:2502.02737*, 2025.

[3] Adithya Bhaskar, Alexander Wettig, Dan Friedman, and Danqi Chen. Finding transformer circuits with edge pruning. In *NeurIPS*, 2024.

[4] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *NeurIPS*, 2023.

[5] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv:2309.08600*, 2023.

[6] Nelson Elhage et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.

[7] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*, 2019.

[8] Leo Gao et al. Weight-sparse transformers have interpretable circuits. *arXiv:2511.13653*, 2025.

[9] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.

[11] Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through l_0 regularization. In *ICLR*, 2018.

[12] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *ICLR*, 2017.

[13] Abhishek Mishra and Krishna Pagare. Prism: Unlocking language model capability extraction. In *1st Conference For AI Scientists (CAISc)*, 2026.

[14] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *ICML*, 2025.

[15] Victor Sanh, Thomas Wolf, and Alexander M. Rush. Movement pruning: Adaptive sparsity by fine-tuning. In *NeurIPS*, 2020.

[16] An Yang et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv:2409.12122*, 2024.

[17] An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.